# OWASP Juice Shop XSS Lab Report

A technical investigation into cross-site scripting (XSS) vulnerabilities in the OWASP Juice Shop application, demonstrating DOM-based and reflected XSS attack vectors with detailed payload analysis and proof-of-concept execution.

Made with GAMMA

# DOM-based XSS Vulnerability

**Attack Vector:** Search functionality via URL fragment parameter

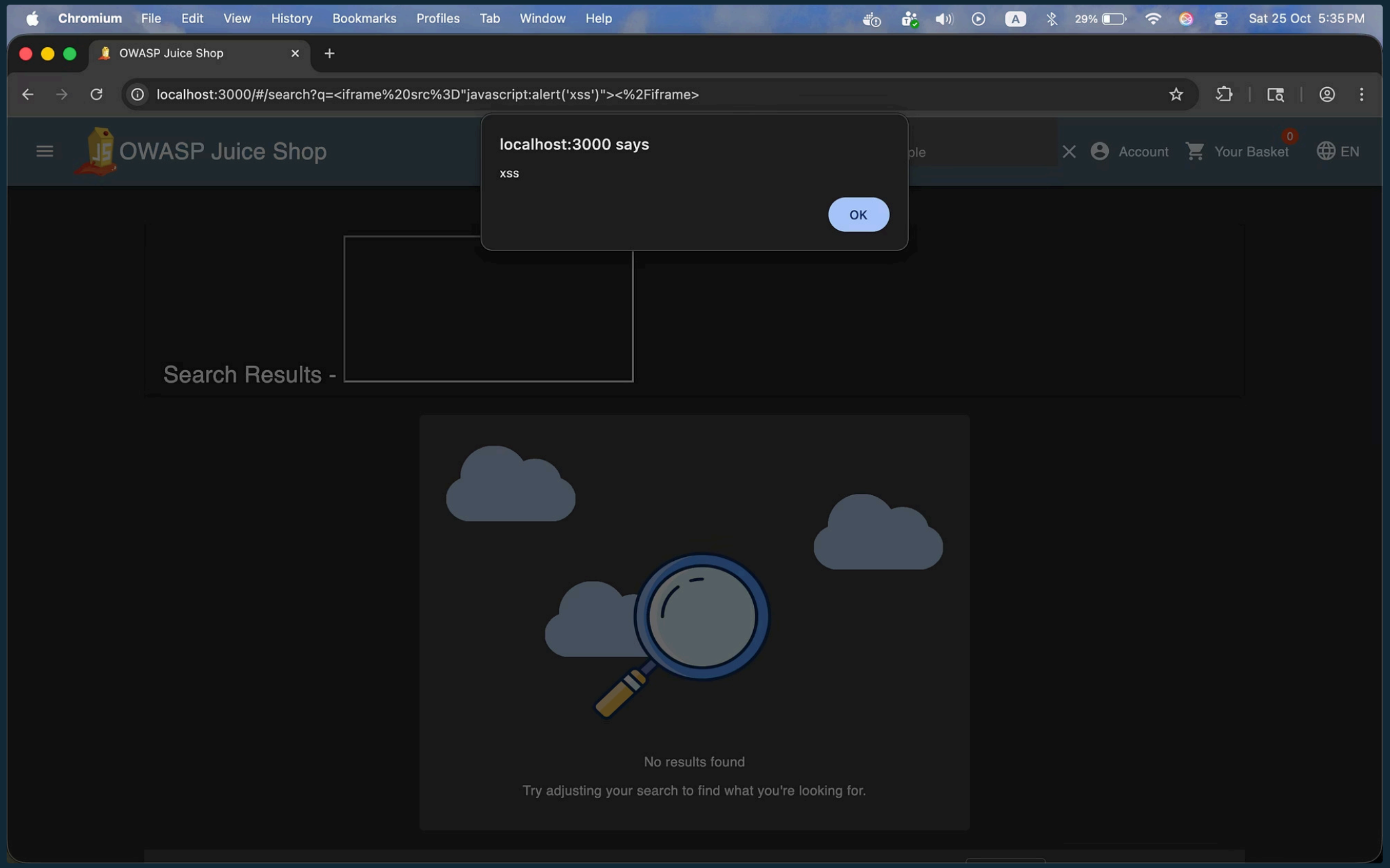
**Payload:** `<iframe src="javascript:alert('xss')"></iframe>`

**URL:** `http://localhost:3000/#/search?q=<iframe src="javascript:alert('xss')"></iframe>`

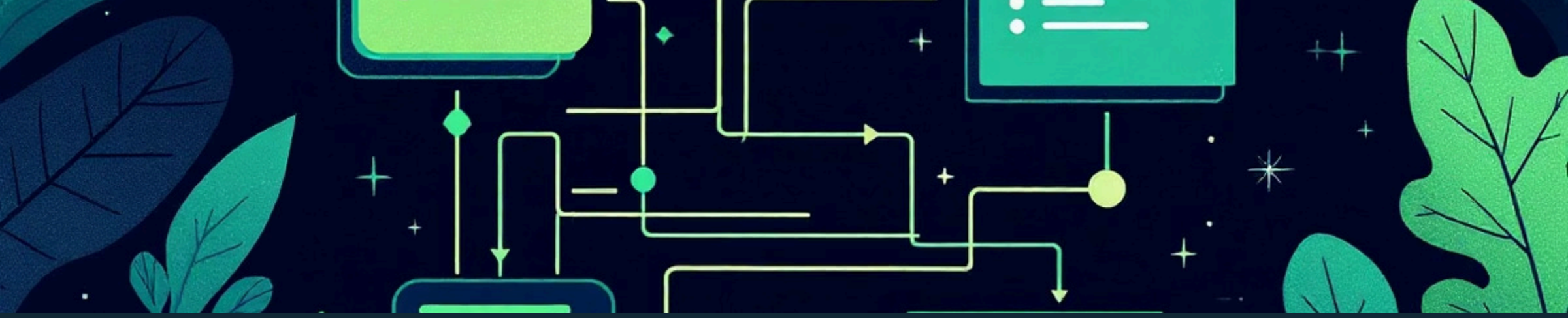
---

The attack exploits client-side JavaScript processing of URL fragments. The payload executes directly in the browser without server transmission, making it invisible to HTTP traffic analysis. Classic DOM-based XSS behavior confirmed through alert box execution.

Made with GAMMA




Made with GAMMA



# Reflected XSS with Backend Constraints

## Challenge

Initial payload with quotes caused backend parser errors in MarsDB when processing URL parameters containing single quotes or double quotes.

## Solution

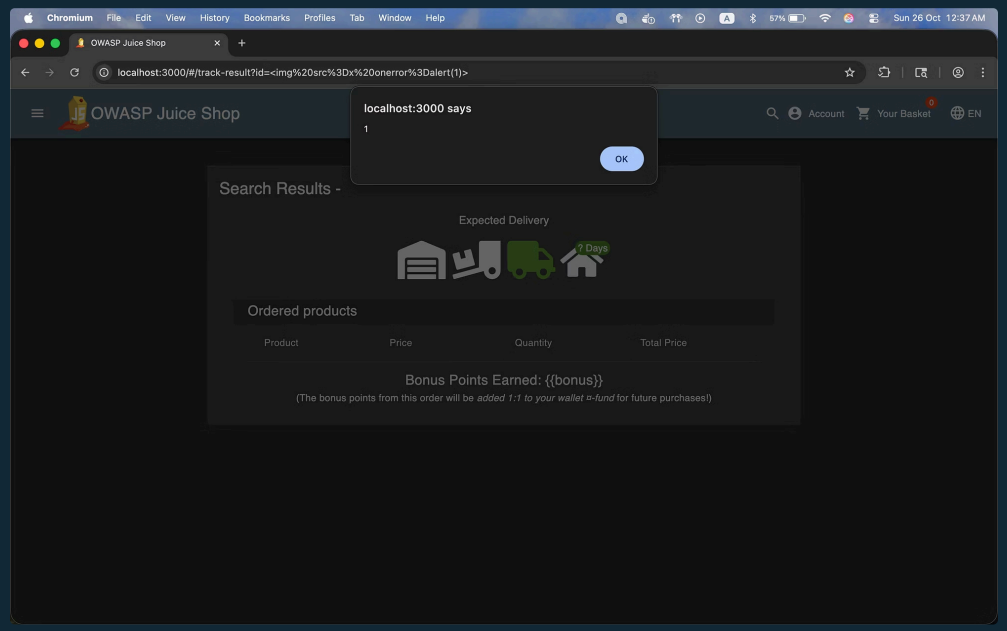
Simplified payload removed quotes: `<img src=x onerror=alert(1)>` successfully bypassed validation and executed in browser.

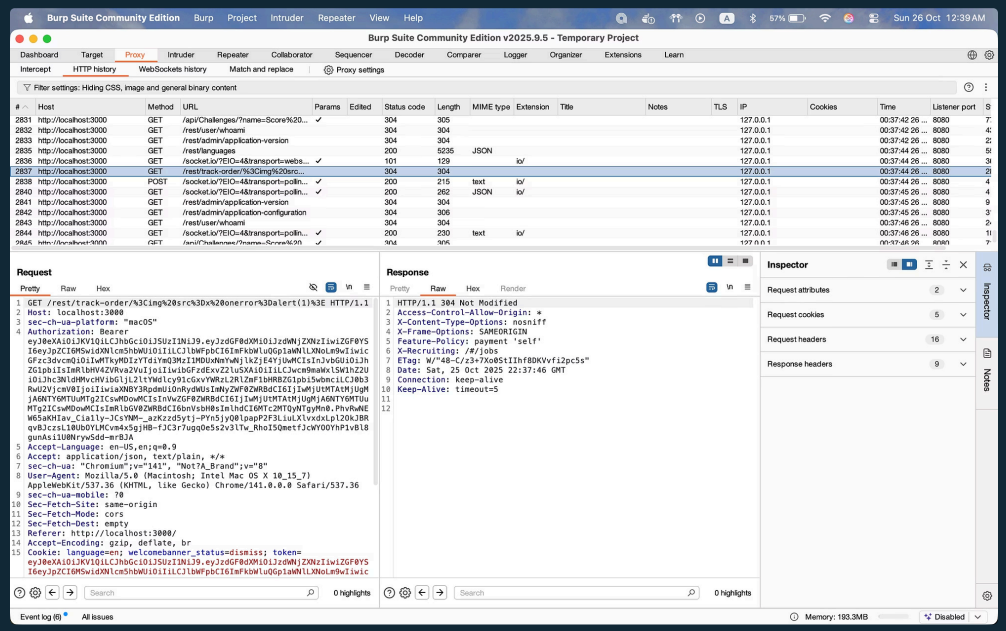
## Confirmation

Server reflected malicious payload in response without sanitization. Alert triggered, confirming reflected XSS through track-order endpoint functionality.

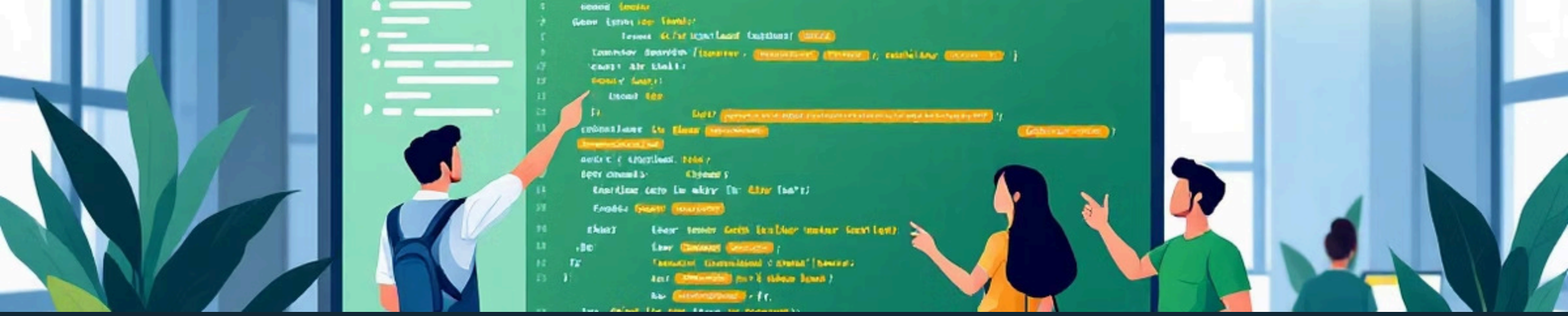
**URL:** http://localhost:3000/#/track-result?id=<img src=x onerror=alert(1)>

Made with GAMMA




Made with GAMMA



# XSS Attack Vectors: Technical Comparison

## DOM-based XSS

Client-side processing. Payload executes via JavaScript handling URL fragments. Invisible to HTTP inspection. No server involvement in execution.

## Reflected XSS

Server processes and reflects payload in HTTP response. Sanitization failures allow browser execution. Observable in request/response traffic via Burp Suite.

## Key Findings

- Both attack vectors successfully executed with visual proof-of-concept evidence captured
- Backend quote parsing restrictions required payload modification to bypass validation
- Vulnerabilities confirmed through browser alerts and HTTP traffic analysis
- Demonstrates critical importance of input sanitization and output encoding across all application layers

Made with GAMMA

# DOM-based vs. Reflected XSS: Architectural Distinctions

Understanding the fundamental differences in how DOM-based and Reflected XSS vulnerabilities manifest is crucial for effective mitigation strategies. While both lead to client-side script execution, their origins and interaction with the server differ significantly.

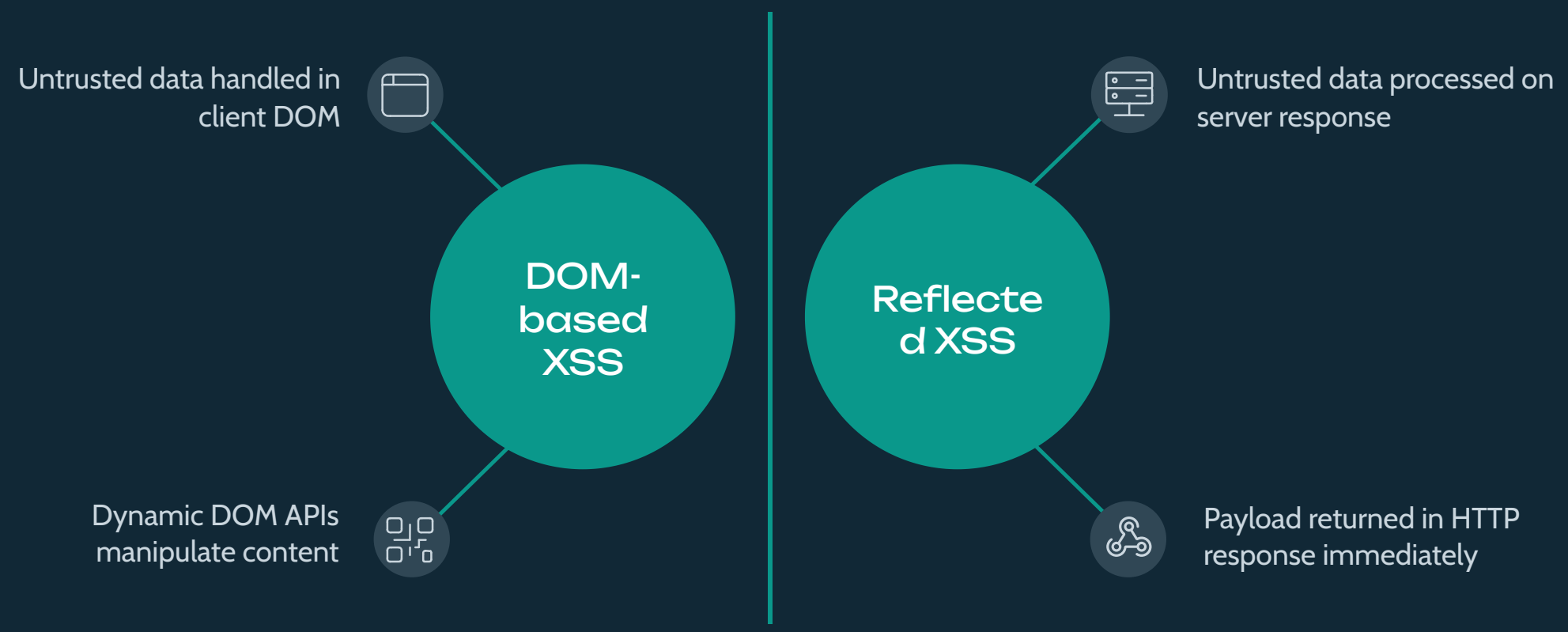
## DOM-based XSS

- Client-side JavaScript reads URL fragment (e.g., `#/search?q=...`).
- JavaScript processes the fragment directly within the browser, without server involvement.
- Malicious payload executes locally before any interaction with the server.
- Term "DOM" refers to JavaScript directly modifying the Document Object Model.

## Reflected XSS

- Server receives the malicious payload as part of an HTTP request.
- Server reflects the payload back in the HTTP response, often due to lack of sanitization.
- Client-side code then renders and inserts the reflected value into the DOM, leading to execution.
- Term "Reflected" indicates the server echoing the untrusted input back to the user's browser.

## Core Architectural Difference




Made with GAMMA